Kien Le

Lab Report

ECE2031 L09

07 March 2026

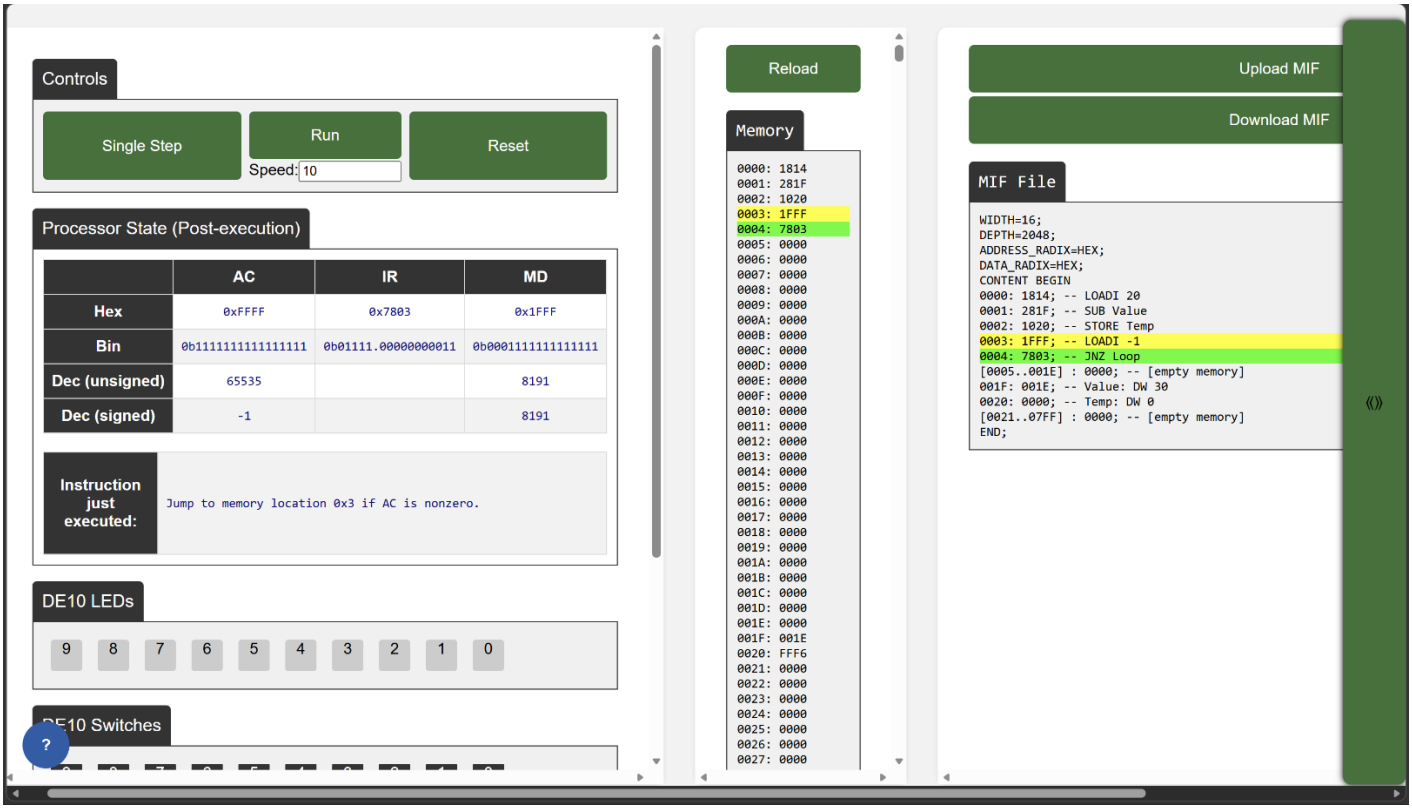

Figure 1. Online simulator with ASM code walking through the load immediate, sub, store, and jump instructions before entering a loop through a condition jump.

```
ORG 0
   ;; Loads 20 into AC then sub the value at mem location value
   LOADI 20
   SUB Value
   STORE Temp

Loop:
   ;; Infinite Loop
   LOADI -1
   JNZ Loop

ORG 31
   ;; Stores the values of variables
   Value: DW 30
   Temp: DW 0
```

Figure 2. ASM code demonstrates the use of subroutines and instructions along with saved memory locations set at desired addresses.

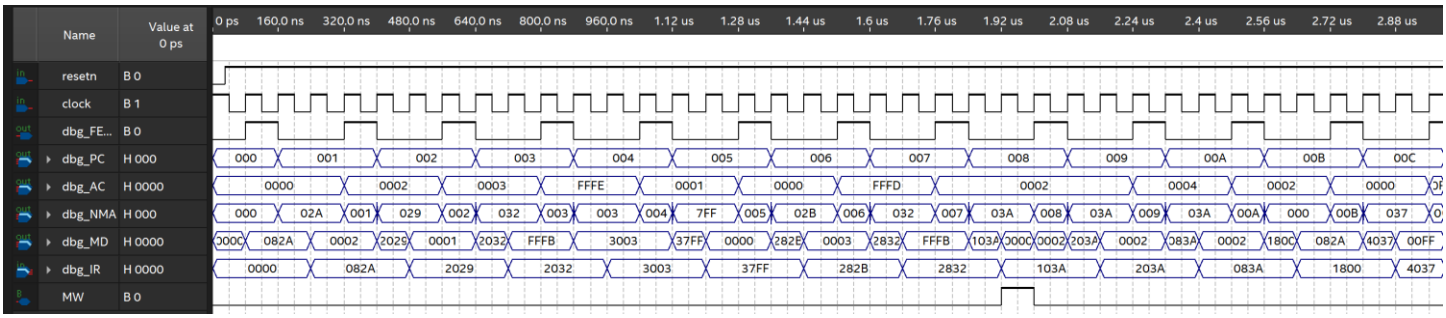

Figure 3. Functional simulation waveform of state machine implementing the SCMOP. The test is running through the instructions within SimpleCode.ASM with this segment specifically executing up to the load immediate of 0 to test the sequence of Adds and Subs.

```
ORG 0
   ;; Loads the value and shifts to get the upper bytes to store
   LOAD Value
   SHIFT -8
   AND Lowmask
   STORE Highbyte
   ;; Loads the value and shifts to get the lower bytes to store
   LOAD Value
   AND Lowmask
   STORE Lowbyte
   ;; Loads back in the saved values and perform the sub op
   LOAD Highbyte
   SUB Lowbyte
   ;; If value is not pos. difference, then we jump to subroutine
   JNEG LowtoHigh
   ;; Else, store result and enter loop
   STORE Result
   JUMP Loop

LowtoHigh:
   ;; Subroutine for reverse subtraction for pos. difference
   LOAD Lowbyte
   SUB Highbyte
   STORE Result
Loop:
   JUMP Loop

ORG 100
   Result: DW 0
ORG 200
   Value: DW 61440
   LowMask: DW 255
   Highbyte: DW 0
   Lowbyte: DW 0
```

Figure 4. ASM code implementing the code to compare the upper and lower bytes of the data at the memory location of value using a mask and bit shift to compare the intended bytes. A check subroutine is implemented to ensure a positive difference for the result value.

Appendix A

Long Code Implementing the State Machine for SCMOP

```vhdl
-- SCOMP.VHD
-- A 16-bit computer designed to be easy to design and modify -- [Added sub, jpos, jnz]
-- Kien Le
-- ECE2031 L09
-- 03/07/2025

library altera_mf;
library lpm;
library ieee;

use altera_mf.altera_mf_components.all;
use lpm.lpm_components.all;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;


entity SCOMP is
   port(
      clock     : in      std_logic;
      resetn    : in      std_logic;
      IO_READ   : out     std_logic;
      IO_WRITE  : out     std_logic;
      IO_ADDR   : out     std_logic_vector(10 downto 0);
      IO_DATA   : inout   std_logic_vector(15 downto 0);
      dbg_FETCH : out     std_logic;
      dbg_AC    : out     std_logic_vector(15 downto 0);
      dbg_PC    : out     std_logic_vector(10 downto 0);
      dbg_NMA   : out     std_logic_vector(10 downto 0);
      dbg_MD    : out     std_logic_vector(15 downto 0);
      dbg_IR    : out     std_logic_vector(15 downto 0)
);
end SCOMP;

architecture a of SCOMP is
   type state_type is (
      init, fetch, decode,
      ex_nop, ex_load, ex_store, ex_store2,
      ex_loadi, ex_add, ex_addi, ex_sub,
      ex_and, ex_or, ex_xor, ex_shift,
      ex_jump, ex_jneg, ex_jpos, ex_jzero, ex_jnz,
      ex_return, ex_call,
      ex_in, ex_in2, ex_out, ex_out2
   );

   -- custom type for the call stack
   constant STACK_SIZE  :  integer := 10;
   type stack_type is array (0 to STACK_SIZE) of std_logic_vector(10 downto 0);

   -- internal signals
   signal state         :  state_type;
   signal AC            :  std_logic_vector(15 downto 0);
   signal AC_shifted    :  std_logic_vector(15 downto 0);
   signal PC_stack      :  stack_type;
   signal IR            :  std_logic_vector(15 downto 0);
   signal mem_data      :  std_logic_vector(15 downto 0);
   signal PC            :  std_logic_vector(10 downto 0);
   signal next_mem_addr :  std_logic_vector(10 downto 0);
   signal operand       :  std_logic_vector(10 downto 0);
   signal MW            :  std_logic;
   signal io_drive_en   :  std_logic;
```

```
begin
   -- use altsyncram component for unified program and data memory
   altsyncram_component : altsyncram
   GENERIC MAP (
      numwords_a => 2048,
      widthad_a => 11,
      width_a => 16,
      init_file => "SimpleDemo.mif",
      clock_enable_output_a => "BYPASS",
      lpm_hint => "ENABLE_RUNTIME_MOD=NO",
      intended_device_family => "CYCLONE V",
      clock_enable_input_a => "BYPASS",
      lpm_type => "altsyncram",
      operation_mode => "SINGLE_PORT",
      power_up_uninitialized => "FALSE",
      read_during_write_mode_port_a => "NEW_DATA_NO_NBE_READ",
      outdata_reg_a => "UNREGISTERED",
      outdata_aclr_a => "NONE",
      width_byteena_a => 1
   )
   PORT MAP (
      wren_a    => MW,
      clock0    => clock,
      address_a => next_mem_addr,
      data_a    => AC,
      q_a       => mem_data
   );

   -- use Intel IP to shift AC for shift instruction
   shifter: lpm_clshift
   generic map (
      lpm_width     => 16,
      lpm_widthdist => 4,
      lpm_shifttype => "arithmetic"
   )
   port map (
      data      => AC,
      distance  => IR(3 downto 0),
      direction => IR(4),
      result    => AC_shifted
   );

   -- Memory address comes from PC during fetch, otherwise from operand
   with state select next_mem_addr <=
      PC when fetch,
      operand when others;

   -- This makes the operand available immediately after fetch
   with state select operand <=
      mem_data(10 downto 0) when decode,
      IR(10 downto 0) when others;

   -- use lpm tri-state driver to drive i/o bus
   io_bus: lpm_bustri
   generic map (
      lpm_width => 16
   )
   port map (
      data     => AC,
      enabledt => io_drive_en,
```

```
   tridata  => IO_DATA
);

-- IR has the IO address during INs and OUTs
IO_ADDR  <= IR(10 downto 0);

process (clock, resetn)
begin
   if (resetn = '0') then          -- Active-low asynchronous reset
      state <= init;
   elsif (rising_edge(clock)) then
      case state is
         when init =>
            MW        <= '0';           -- clear memory write flag
            PC        <= "00000000000"; -- reset PC to the beginning of memory, address 0x000
            AC        <= x"0000";       -- clear AC register
            io_drive_en <= '0';         -- don't drive IO
            state     <= fetch;         -- start fetch-decode-execute cycle

         when fetch =>
            io_drive_en <= '0';    -- stop driving IO data after an out
            PC        <= PC + 1;   -- increment PC to next instruction address
            state     <= decode;

         when decode =>
            IR    <= mem_data;          -- latch instruction into the IR
            case mem_data(15 downto 11) is -- opcode is top 5 bits of instruction
               when "00000"  =>        -- no operation (nop)
                  state <= ex_nop;
               when "00001"  =>        -- load
                  state <= ex_load;
               when "00010"  =>        -- store
                  MW    <= '1';        -- initiate a memory write
                  state <= ex_store;
               when "00011"  =>        -- loadi
                  state <= ex_loadi;
               when "00100"  =>        -- add
                  state <= ex_add;
               when "00101"  =>        -- sub
                  state <= ex_sub;
               when "00110"  =>        -- addi
                  state <= ex_addi;
               when "00111"  =>        -- and
                  state <= ex_and;
               when "01000"  =>        -- or
                  state <= ex_or;
               when "01001"  =>        -- xor
                  state <= ex_xor;
               when "01010"  =>        -- shift
                  state <= ex_shift;
               when "01011"  =>        -- jump
                  state <= ex_jump;
               when "01100"  =>        -- jneg
                  state <= ex_jneg;
               when "01101"  =>        -- jpos
                  state <= ex_jpos;
               when "01110"  =>        -- jzero
                  state <= ex_jzero;
               when "01111"  =>        -- jnz
                  state <= ex_jnz;
               when "10000"  =>        -- call
```

```
            state <= ex_call;
         when "10001"  =>        -- return
            state <= ex_return;
         when "10010"  =>        -- in
            state <= ex_in;
         when "10011"  =>        -- out
            state <= ex_out;
            io_drive_en <= '1';  -- start driving IO data
         when others =>
            state <= ex_nop;     -- invalid opcodes are nop
      end case;

   when ex_nop =>
      state <= fetch;

   when ex_load =>
      AC    <= mem_data;     -- latch data from mem_data (memory contents) to AC
      state <= fetch;

   when ex_store =>
      MW    <= '0';          -- drop MW to end write cycle
      state <= fetch;

   when ex_loadi =>             -- sign-extend the immediate
      AC    <= (IR(10) & IR(10) & IR(10) &
       IR(10) & IR(10) & IR(10 downto 0));
      state <= fetch;

   when ex_add =>
      AC    <= AC + mem_data;   -- addition
      state <= fetch;

   when ex_addi =>
      -- sign extension
      AC    <= AC + (IR(10) & IR(10) & IR(10) &
       IR(10) & IR(10) & IR(10 downto 0));
      state <= fetch;

   when ex_jump =>
      PC    <= operand;         -- overwrite PC with new address
      state <= fetch;

   when ex_jneg =>
      if (AC(15) = '1') then
         PC    <= operand;      -- Change the program counter to the operand
      end if;
      state <= fetch;

   when ex_jpos =>
      if (AC(15) = '0' and AC /= x"0000") then
         PC    <= operand;   -- Change the program counter to the operand
      end if;
      state <= fetch;
   when ex_jzero =>

      if (AC = x"0000") then
         PC    <= operand;
      end if;
      state <= fetch;

   when ex_jnz =>
```

```
        if (AC /= x"0000") then
            PC    <= operand;
        end if;
        state <= fetch;

    when ex_and =>
        AC    <= AC and mem_data; -- logical bitwise AND
        state <= fetch;

    when ex_sub =>
        AC     <= AC - mem_data;
        state <= fetch;

    when ex_or =>
        AC    <= AC or mem_data;
        state <= fetch;

    when ex_xor =>
        AC    <= AC xor mem_data;
        state <= fetch;

    when ex_shift =>
        AC    <= AC_shifted;    -- shift is accomplished with a dedicated shifter
        state <= fetch;

    when ex_call =>
        for i in 0 to STACK_SIZE-1 loop
            PC_stack(i + 1) <= PC_stack(i);
        end loop;
        PC_stack(0) <= PC;
        PC          <= operand;
        state       <= fetch;

    when ex_return =>
        for i in 0 to STACK_SIZE-1 loop
            PC_stack(i) <= PC_stack(i + 1);
        end loop;
        PC          <= PC_stack(0);
        state       <= fetch;

    when ex_in =>
        IO_READ <= '1';  -- instruct peripheral to drive bus
        state <= ex_in2;

    when ex_in2 =>
        IO_READ <= '0';
        AC <= IO_DATA;   -- latch in data from peripheral
        state <= fetch;

    when ex_out =>
        IO_WRITE <= '1'; -- tell peripheral that data is present
        state <= ex_out2;

    when ex_out2 =>
        IO_WRITE <= '0';
        state <= fetch;

    when others =>
        state <= init;   -- if an invalid state is reached, reset

end case;
```

```
      end if;
   end process;

   -- Additional outputs to aid simulation
      dbg_FETCH <= '1' when state = fetch else '0';
   dbg_PC    <= PC;
   dbg_AC    <= AC;
   dbg_IR    <= IR;
   dbg_NMA   <= next_mem_addr;
   dbg_MD    <= mem_data;

end a;
```